

Job Search Kit: getting started

This guide takes you from a bare computer to the moment the kit's own setup wizard takes over. Nothing here is difficult, and none of it needs programming knowledge. Allow half an hour.

What you are setting up

The Job Search Kit is a private, AI-operated working environment for a job search. Once it is installed, an AI assistant works inside it with you: assessing whether roles are genuinely worth your time, tailoring your CV to each one, drafting cover letters in your voice, prepping you for interviews, and keeping an application log so nothing goes cold. Everything lives in a folder on your machine, and it never applies to anything on your behalf.

What you need

- A computer running **macOS**, **Linux**, or **Windows** (Windows works through WSL, step 1 below).
- An **AI coding assistant**. The kit is built for **Claude Code** and works best with it, which needs a paid Claude plan (Pro or Max). Codex, Gemini CLI and Cursor can also run your search, each under its own plan, with some differences worth knowing about before you choose. See “Using a different AI assistant” near the end of this guide.
- Your **current CV** and links to your portfolio or LinkedIn, for the first session after setup.

Step 1: open a terminal

The terminal is where you will paste a handful of commands. That is the extent of it. One thing to know before you start: when a grey box in this guide shows more than one line, each line is a separate command. Paste the first line, press Enter, wait for it to finish, then paste the next.

- **macOS**: open **Terminal** (Spotlight: press Cmd+Space, type “terminal”).
- **Linux**: you already know where it is.
- **Windows**: the kit needs WSL (Windows Subsystem for Linux). Open **PowerShell as administrator** and run:

```
wsl --install
```

Restart when prompted, open the new **Ubuntu** app from the Start menu, and do the remaining steps inside it.

Step 2: install git

Git fetches the kit and keeps a history of your project. Check whether you already have it:

```
git --version
```

If that prints a version number, move on. If not:

- **macOS**: run `xcode-select --install` and accept the prompt.
- **Linux / WSL**: run `sudo apt update && sudo apt install -y git`

Step 3: install your AI assistant

For Claude Code, run the installer, then start it (two commands, one at a time):

```
curl -fsSL https://claude.ai/install.sh | bash
claude
```

The first launch walks you through logging in with your Claude account. When you can type a question and get an answer, you are done here: type `/exit` to close Claude Code and return to the terminal. You will not need it again until the wizard hands back to it at the end of setup.

If you prefer a different assistant, install it per its own instructions. During setup the wizard will ask which tool you use and prepare the project accordingly.

Step 4 (optional): Node.js, for the portal

The kit includes an optional web portal that lets you submit job adverts from your phone and receive tailored PDFs by email. It needs Node.js 18 or newer. Skip this happily; you can add it later.

- **macOS:** `brew install node` (or the installer from nodejs.org)
- **Linux / WSL:** `sudo apt install -y nodejs npm`

Step 5: fetch the kit and hand over to the wizard

First, choose a folder for the kit. It, and everything it makes for your job search, lives in one folder. The simplest choice is a new folder called Jobs in your home folder. Back in the terminal, run these two commands, one at a time:

```
mkdir -p ~/Jobs
cd ~/Jobs
```

(~ means your home folder. The first command makes the Jobs folder, and does nothing if it is already there; the second moves you into it.) If you would rather use a different folder, type `cd` with a space after it, drag that folder from Finder or your file manager into the terminal window, and press Enter.

From here the kit guides you itself. Still in the terminal, run these two commands, one at a time (the first downloads the kit into a j4 folder inside the one you chose, the second starts the wizard):

```
git clone https://github.com/spacetimejam/j4
cd j4 && ./setup/setup.sh
```

The wizard checks your machine, asks a short set of questions about you and your search (including whether to register you with the shared submission portal, if your household runs one, and if so whether you should receive failure alerts from it), and builds your personal project folder inside the kit, named with your initials (for example j4/sj). When it finishes, it tells you the final step: move into that new folder in the terminal and start your AI assistant from inside it, for example `cd ~/Jobs/j4/sj` and then `claude`. Starting it from inside the folder is what lets the assistant see your project. Once it is running, say “**run setup**”. The assistant then interviews you properly, builds your profile and CV materials, and from that point on you are running your search together.

Using a different AI assistant

The kit is built for Claude Code and that is the smoothest path. Three other assistants can run your search too. What differs is not the quality of the help you get day to day, but whether the optional web portal works with them.

Claude Code. Everything works: your project folder and the portal. Nothing extra to do.

Codex. Your project folder works straight away. Portal support is new. To install it (Node.js from step 4 is needed first):

```
npm install -g @openai/codex
codex
```

The first launch walks you through signing in. To use it with the portal, set `AGENT_RUNNER=codex` in the portal's configuration, and ask whoever runs the portal to run its calibration check first. That check is described in `docs/portal.md`; it takes a couple of minutes and confirms the kit and your version of Codex agree with each other. Note that `AGENT_RUNNER` applies to the whole portal, not just your login: on a shared household portal, setting it to `codex` switches every user's sessions to Codex, including anyone using Claude Code with only a `CLAUDE.md` in their project folder. Agree this with whoever else uses the portal before changing it.

Gemini CLI. Your project folder works, after one setting. To install it:

```
npm install -g @google/gemini-cli
gemini
```

Gemini looks for a file called `GEMINI.md`, while the kit writes `AGENTS.md`. Point it at the right file by creating `.gemini/settings.json` in your project folder containing:

```
{ "contextFileName": ["AGENTS.md", "GEMINI.md"] }
```

On newer versions that setting is nested instead, as `{ "context": { "fileName": ["AGENTS.md", "GEMINI.md"] } }`. If one form is ignored, try the other. There is no portal support for Gemini CLI.

Cursor. Your project folder works straight away: Cursor reads `AGENTS.md` without configuration. There is no portal support for Cursor.

The portal is optional in all cases. Without it you submit job adverts by talking to your assistant in the terminal, which is how the kit is designed to work anyway; the portal exists so you can also do it from your phone.

Good to know

- **Everything stays on your machine.** The project folder contains your CV, salary expectations and application history. Treat it as private, and back it up as you would any personal documents.
- **The kit never applies for you.** Every application is tailored, checked by you, and sent by you. Quality over volume is the whole philosophy.
- **AI sessions cost money** in the usual way your Claude (or other) plan meters usage. A tailored application is a normal working session, not a one-line question.
- **Stuck?** Ask the person who sent you this guide, or ask your AI assistant itself: once the kit is installed it knows how everything fits together.